Database Management System (DBMS) Operations:

1. Create Database:

Purpose: Create a new database.

Command:

```
CREATE DATABASE CollegeDB;
```

Output:

Database CollegeDB is created.

2. Select Database:

Purpose: Choose database to work.

Command:

```
USE CollegeDB;
```

Output:

Database changed

3. Create Table:

Purpose: Create table to store data.

Command:

```
CREATE TABLE Student(
USN INT,
Name VARCHAR(20),     //(Varchar- variable character.It is used to store text or characters)
Branch VARCHAR(20),
Marks INT);
```

Output:

Query OK, 0 rows affected

4. Describe Table:

Purpose: Show table structure.

Command:

DESC Student;

Output:

Field	Type	Null	Key	Default	Extra
USN	int	YES		NULL	
Name	Varchar(20)	YES		NULL	
Branch	Varchar(20)	YES		NULL	
Marks	int	YES		NULL	

5. Insert Records:

Purpose: Add data into table.

Command:

```
INSERT INTO Student VALUES(1,'Ravi','CSE',85);
INSERT INTO Student VALUES(2,'Anita','ECE',90);
INSERT INTO Student VALUES(3,'Rahul','CSE',75);
INSERT INTO Student VALUES(4,'Sneha','EEE',88);
```

Output:

Query OK, 1 row affected

6. Display All Records:

Purpose: Retrieve data. //Retrieve means displaying data from a database tables

Command:

```
SELECT * FROM Student;
```

Output:

USN	Name	Branch	Marks
1	Ravi	CSE	85
2	Anita	ECE	90
3	Rahul	CSE	75
4	Sneha	EEE	88

7. SELECT with Condition (WHERE): It is used to filter data based on a condition. **(or)**

Purpose: The WHERE clause is used to:

- Filter data from a table
- Retrieve specific rows instead of all rows
- Apply conditions such as equal, greater than, less than, etc.
- Search particular records based on values
- Reduce unnecessary data display

Command:

```
SELECT * FROM Student
WHERE Branch='CSE';
```

Output:

USN	Name	Branch	Marks
1	Ravi	CSE	85
3	Rahul	CSE	75

8. UPDATE Records:

Purpose: Change or Modify existing data in a table.

Command:

```
UPDATE Student
SET Marks=95
WHERE USN=2;
```

Output:

Query OK, 1 row affected

Check:

```
SELECT * FROM Student;
```

Output:

USN	Name	Branch	Marks
1	Ravi	CSE	85
2	Anita	ECE	95
3	Rahul	CSE	75

4	Sneha	EEE	88

9. DELETE Record:

Purpose: Remove specific record.

Command:

```
DELETE FROM Student
WHERE USN=3;
```

Output:

Query OK, 1 row affected

Table After Delete:

USN	Name	Branch	Marks
1	Ravi	CSE	85
2	Anita	ECE	95
4	Sneha	EEE	88

10. ORDER BY:

Purpose: Sort records.

Command:

```
SELECT * FROM Student
 ORDER BY Marks DESC;
```

Output:

USN	Name	Branch	Marks
2	Anita	ECE	95
4	Sneha	EEE	88
1	Ravi	CSE	85

11. GROUP BY:

Purpose: The GROUP BY clause is used to:

- To group similar records together
- To count or calculate values for each group

- To summarize data

Command:

```
SELECT Branch, COUNT(*)
FROM Student
GROUP BY Branch;
```

Output:

Branch	COUNT(*)
CSE	1
ECE	1
EEE	1

12. DISTINCT:

Purpose:

- To display unique records
- To remove duplicate values
- To show different values in a column

Command:

```
SELECT DISTINCT Branch FROM Student;
```

Output:

Branch
CSE
ECE
EEE

13. LIKE (Pattern Search):

Purpose: The LIKE operator is used to search for a specific pattern in a column. It is mainly used to:

- Find records that match a pattern
- Search names or text starting with or ending with certain letters
- Filter text data

Command:

```
SELECT * FROM Student
WHERE Name LIKE 'R%';
```

Output:

USN	Name	Branch	Marks
1	Ravi	CSE	85

14. ALTER TABLE:

Purpose: The ALTER TABLE command is used to change the structure of an existing table.

It is used to:

- Add a new column
- Delete a column
- Modify a column datatype
- Rename a column

Command:

```
ALTER TABLE Student
ADD Phone VARCHAR(10);
```

Output:

Query OK, 0 rows affected

Structure After ALTER:

Field	Type
USN	int
Name	Varchar(20)
Branch	Varchar(20)
Marks	int
Phone	Varchar(10)

15. TRUNCATE:

Purpose: The TRUNCATE command is used to delete all records from a table quickly, but the table structure remains the same. It is used to:

- Remove all rows from a table

- Clear table data quickly
- Keep the table structure for future use

Command:

```
TRUNCATE TABLE Student;
```

Output:

Query OK, 0 rows affected

Result:

Table becomes empty.

16. DROP TABLE:

Purpose: The DROP TABLE command is used to delete the entire table from the database.

It will:

- Remove the table structure
- Delete all records in the table
- Completely remove the table from the database

Command:

```
DROP TABLE Student;
```

Output:

Query OK, 0 rows affected

Table is removed from database.

PROGRAM 1:

Consider the schema for College Database:

STUDENT(USN, SName, Address, Phone, Gender)

SEMSEC(SSID, Sem, Sec)

CLASS(USN, SSID)

COURSE(Subcode, Title, Sem, Credits)

IAMARKS(USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Write SQL queries to :

1. List all the student details studying in fourth semester 'C' section.
2. Compute the total number of male and female students in each semester and in each section.
3. Create a view of Test1 marks of student USN '4MH22CS200' in all Courses.
4. Calculate the FinalIA (average of best two test marks) and update the corresponding table for all students.
5. Categorize students based on the following criterion:

   If FinalIA = 17 to 20 then CAT = 'Outstanding'

   If FinalIA = 12 to 16 then CAT = 'Average'

   If FinalIA< 12 then CAT = 'Weak'

Give these details only for 4th semester ALL section students.

Step 1: Create Database:

CREATE DATABASE COLLEGE_DB;

Output:

Query OK, 1 row affected

Use the database:

USE COLLEGE_DB;

Output:

Database changed

Step 2: Create Tables:

1. STUDENT Table

```
CREATE TABLE STUDENT(
USN VARCHAR(10) PRIMARY KEY,
SName VARCHAR(30),
Address VARCHAR(50),
Phone VARCHAR(10),
Gender VARCHAR(10)
);
```

Output:

Query OK, 0 rows affected

Check structure:

DESC STUDENT;

Field	Type	Key
USN	Varchar(10)	PRI
SName	Varchar(30)	
Address	Varchar(50)	
Phone	Varchar(10)	
Gender	Varchar(10)	

2. SEMSEC Table

```
CREATE TABLE SEMSEC(
SSID VARCHAR(5) PRIMARY KEY,
Sem INT,
Sec CHAR(1)
);
```

Output:

Query OK, 0 rows affected

Check structure:

DESC SEMSEC;

3. CLASS Table

```
CREATE TABLE CLASS(
USN VARCHAR(10),
SSID VARCHAR(5),
PRIMARY KEY(USN, SSID),
FOREIGN KEY (USN) REFERENCES STUDENT(USN),
FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

Output:

Query OK, 0 rows affected

Check structure:

```
DESC CLASS;
```

4. COURSE Table

```
CREATE TABLE COURSE(
Subcode VARCHAR(10) PRIMARY KEY,
Title VARCHAR(30),
Sem INT,
Credits INT
);
```

Output:

Query OK, 0 rows affected

Check structure:

```
DESC COURSE;
```

5. IAMARKS Table

```
CREATE TABLE IAMARKS(
USN VARCHAR(10),
```

```
Subcode VARCHAR(10),
SSID VARCHAR(5),
Test1 INT,
Test2 INT,
Test3 INT,
FinalIA INT,
PRIMARY KEY(USN, Subcode, SSID),
FOREIGN KEY (USN) REFERENCES STUDENT(USN),
FOREIGN KEY (Subcode) REFERENCES COURSE(Subcode),
FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

Output:

Query OK, 0 rows affected

Check structure:

```
DESC IAMARKS;
```

Step 3: Insert Sample Data:

1. Insert STUDENT

```
INSERT INTO STUDENT VALUES
('4MH22CS200','Rahul','Bangalore','9876543210','Male'),
('4MH22CS201','Anita','Mysore','9876543211','Female'),
('4MH22CS202','Ravi','Tumkur','9876543212','Male'),
('4MH22CS203','Pooja','Bangalore','9876543213','Female');
```

Output:

Query OK, 4 rows affected

Check inserted data:

```
SELECT * FROM STUDENT;
```

USN	SName	Address	Phone	Gender
4MH22CS200	Rahul	Bangalore	9876543210	Male
4MH22CS201	Anita	Mysore	9876543211	Female
4MH22CS202	Ravi	Tumkur	9876543212	Male
4MH22CS203	Pooja	Bangalore	9876543213	Female

2. Insert SEMSEC

INSERT INTO SEMSEC VALUES

('S1',4,'C'),

('S2',4,'A'),

('S3',5,'B');

Output:

Query OK, 3 rows affected

Check inserted data:

SELECT * FROM SEMSEC;

SSID	Sem	Sec
S1	4	C
S2	4	A
S3	5	B

3. Insert Data into CLASS Table

INSERT INTO CLASS VALUES

('4MH22CS200','S1'),

('4MH22CS201','S1'),

('4MH22CS202','S2'),

('4MH22CS203','S1');

Output:

Query OK, 4 rows affected

Check inserted data:

SELECT * FROM CLASS;

USN	SSID
4MH22CS200	S1
4MH22CS201	S1
4MH22CS202	S2
4MH22CS203	S1

4. Insert Data into COURSE Table

INSERT INTO COURSE VALUES

('CS401','DBMS',4,4),

('CS402','Operating System',4,4),

('CS403','Computer Networks',4,3);

Output:

Query OK, 3 rows affected

Check inserted data:

SELECT * FROM COURSE;

Subcode	Title	Sem	Credits
CS401	DBMS	4	4
CS402	Operating System	4	4
CS403	Computer Networks	4	3

5. Insert Data into IAMARKS Table

INSERT INTO IAMARKS VALUES

('4MH22CS200','CS401','S1',18,19,20,NULL),

('4MH22CS200','CS402','S1',15,17,16,NULL),

('4MH22CS201','CS401','S1',10,12,14,NULL),

('4MH22CS203','CS403','S1',8,10,9,NULL);

Output:

Query OK, 4 rows affected

Check inserted data:

SELECT * FROM IAMARKS;

USN	Subcode	SSID	Test1	Test2	Test3	FinalIA
4MH22CS200	CS401	S1	18	19	20	NULL
4MH22CS200	CS402	S1	15	17	16	NULL
4MH22CS201	CS401	S1	10	12	14	NULL
4MH22CS203	CS403	S1	8	10	9	NULL

Step 4: SQL Queries:

1. List all the student details studying in fourth semester ‘C’ section.

```
SELECT S.*
FROM STUDENT S
JOIN CLASS C ON S.USN = C.USN
JOIN SEMSEC SS ON C.SSID = SS.SSID
WHERE SS.Sem = 4 AND SS.Sec = 'C';
```

Output:

USN	SName	Address	Phone	Gender
4MH22CS200	Rahul	Bangalore	9876543210	Male
4MH22CS201	Anita	Mysore	9876543211	Female
4MH22CS203	Pooja	Bangalore	9876543213	Female

2. Compute the total number of male and female students in each semester and in each section.

```
SELECT SS.Sem, SS.Sec, S.Gender, COUNT(*) AS Total
FROM STUDENT S
JOIN CLASS C ON S.USN = C.USN
JOIN SEMSEC SS ON C.SSID = SS.SSID
GROUP BY SS.Sem, SS.Sec, S.Gender;
```

Output:

Sem	Sec	Gender	Total
4	C	Male	1
4	C	Female	2
4	A	Male	1

3. Create a view of Test1 marks of student USN ‘4MH22CS200’ in all Courses.

```
CREATE VIEW TEST1_MARKS AS
SELECT Subcode, Test1
FROM IAMARKS
WHERE USN = '4MH22CS200';
```

View result:

```
SELECT * FROM TEST1_MARKS;
```

Output:

Subcode	Test1
CS401	18
CS402	15

4. Calculate the FinalIA (average of best two test marks) and update the corresponding table for all students.

```
UPDATE IAMARKS
SET FinalIA =
(GREATEST(Test1,Test2) +
GREATEST(Test2,Test3) +
GREATEST(Test1,Test3)
-
LEAST(Test1,Test2,Test3)) / 2;
```

Check table:

SELECT * FROM IAMARKS;

Output:

USN	Subcode	Test1	Test2	Test3	FinalIA
4MH22CS200	CS401	18	19	20	19
4MH22CS200	CS402	15	17	16	16
4MH22CS201	CS401	10	12	14	13
4MH22CS203	CS403	8	10	9	9

5. Categorize students based on the following

Criterion: If FinalIA = 17 to 20 then CAT = ‘Outstanding’

If FinalIA = 12 to 16 then CAT = ‘Average’

If FinalIA< 12 then CAT = ‘Weak’

Give these details only for 4th semester ALL section students.

```
SELECT S.USN, S.SName, SS.Sem, SS.Sec, I.FinalIA,
CASE
WHEN FinalIA BETWEEN 17 AND 20 THEN 'Outstanding'
WHEN FinalIA BETWEEN 12 AND 16 THEN 'Average'
WHEN FinalIA < 12 THEN 'Weak'
END AS Category
FROM STUDENT S
JOIN CLASS C ON S.USN=C.USN
JOIN SEMSEC SS ON C.SSID=SS.SSID
JOIN IAMARKS I ON S.USN=I.USN
WHERE SS.Sem=4;
```

Output:

USN	Name	Sem	Sec	FinalIA	Category
4MH22CS200	Rahul	4	C	19	Outstanding
4MH22CS200	Rahul	4	C	16	Average

4MH22CS201	Anita	4	C	13	Average
4MH22CS203	Pooja	4	C	9	Weak

PROGRAM 2:

Consider the schema for Movie Database:

ACTOR(Act_id, Act_Name, Act_Gender)

DIRECTOR(Dir_id, Dir_Name, Dir_Phone)

MOVIES(Mov_id, Mov_Title, Mov_Year, Mov_Lang, Dir_id)

MOVIE_CAST(Act_id, Mov_id, Role)

RATING(Mov_id, Rev_Stars)

Write SQL queries to

1. List the titles of all movies directed by 'Mani Rathnam'.
2. Find the movie names where one or more actors acted in two or more movies.
3. List all actors who acted in a movie before 2000 and also in a movie after 2024(use JOIN operation).
4. Find the title of movies and number of stars for each movie that has at least one rating and find the highest number of stars that movie received. Sort the result by movie title.

Update rating of all movies directed by 'Shankar' to 5.

Step 1: Create Database:

CREATE DATABASE MOVIE_DB;

Output:

Query OK, 1 row affected

Use the database:

USE MOVIE_DB;

Output:

Database changed

Step 2: Create Tables:

1. ACTOR Table

CREATE TABLE ACTOR(

Act_id INT PRIMARY KEY,

Act_Name VARCHAR(30),

Act_Gender VARCHAR(10)

);

Output:

Query OK, 0 rows affected

Check structure:

DESC ACTOR;

Output:

Field	Type	Key
Act_id	int	PRI
Act_Name	varchar(30)	
Act_Gender	varchar(10)	

2. DIRECTOR Table

CREATE TABLE DIRECTOR(

Dir_id INT PRIMARY KEY,

Dir_Name VARCHAR(30),

Dir_Phone VARCHAR(15)

);

Output:

Query OK, 0 rows affected

Check structure:

DESC DIRECTOR;

3. MOVIES Table

CREATE TABLE MOVIES(

Mov_id INT PRIMARY KEY,

Mov_Title VARCHAR(40),

Mov_Year INT,

```
Mov_Lang VARCHAR(20),
Dir_id INT,
FOREIGN KEY (Dir_id) REFERENCES DIRECTOR(Dir_id)
);
```

Output:

Query OK, 0 rows affected

Check structure:

```
DESC MOVIES;
```

4. MOVIE_CAST Table

```
CREATE TABLE MOVIE_CAST(
Act_id INT,
Mov_id INT,
Role VARCHAR(30),
PRIMARY KEY(Act_id,Mov_id),
FOREIGN KEY (Act_id) REFERENCES ACTOR(Act_id),
FOREIGN KEY (Mov_id) REFERENCES MOVIES(Mov_id)
);
```

Output:

Query OK, 0 rows affected

Check structure:

```
DESC MOVIE_CAST;
```

5. RATING Table

```
CREATE TABLE RATING(
Mov_id INT,
Rev_Stars INT,
FOREIGN KEY (Mov_id) REFERENCES MOVIES(Mov_id));
```

Output:

Query OK, 0 rows affected

Check structure:

DESC RATING;

Step 3: Insert Sample Data:

1. Insert into ACTOR

INSERT INTO ACTOR VALUES

(1,'Rajinikanth','Male'),

(2,'Kamal Haasan','Male'),

(3,'Aishwarya Rai','Female'),

(4,'Trisha','Female');

Output:

Query OK, 4 rows affected

Check structure:

SELECT * FROM ACTOR;

Output:

Act_id	Act_Name	Act_Gender
1	Rajinikanth	Male
2	Kamal Haasan	Male
3	Aishwarya Rai	Female
4	Trisha	Female

2. Insert into DIRECTOR

INSERT INTO DIRECTOR VALUES

(10,'Mani Rathnam','9876543210'),

(11,'Shankar','9876543211'),

(12,'Lokesh','9876543212');

Output:

Query OK, 3 rows affected

Check structure:

SELECT * FROM DIRECTOR;

Output:

Dir_id	Dir_Name	Dir_Phone
10	Mani Rathnam	9876543210
11	Shankar	9876543211
12	Lokesh	9876543212

3. Insert into MOVIES

INSERT INTO MOVIES VALUES

(100,'Roja',1992,'Tamil',10),

(101,'Bombay',1995,'Tamil',10),

(102,'Robot',2010,'Tamil',11),

(103,'Sivaji',2007,'Tamil',11),

(104,'Future Film',2025,'Tamil',12);

Output:

Query OK, 5 rows affected

Check structure:

SELECT * FROM MOVIES;

Output:

Mov_id	Mov_Title	Mov_Year	Mov_Lang	Dir_id
100	Roja	1992	Tamil	10
101	Bombay	1995	Tamil	10
102	Robot	2010	Tamil	11
103	Sivaji	2007	Tamil	11
104	Future Film	2025	Tamil	12

4. Insert into MOVIE_CAST

INSERT INTO MOVIE_CAST VALUES

(1,102,'Hero'),

(1,103,'Hero'),

(2,100,'Hero'),

(3,101,'Heroine'),

(3,104,'Heroine');

Output:

Query OK, 5 rows affected

Check structure:

SELECT * FROM MOVIE_CAST;

Output:

Act_id	Mov_id	Role
1	102	Hero
1	103	Hero
2	100	Hero
3	101	Heroine
3	104	Heroine

5. Insert into RATING

INSERT INTO RATING VALUES

(100,4),

(101,5),

(102,3),

(103,4),

(104,2);

Output:

Query OK, 5 rows affected

Check structure:

SELECT * FROM RATING;

Output:

Mov_id	Rev_Stars
100	4
101	5
102	3
103	4
104	2

Step 4: SQL Queries:

1. List the titles of all movies directed by ‘Mani Rathnam’.

```
SELECT M.Mov_Title
FROM MOVIES M
JOIN DIRECTOR D
ON M.Dir_id = D.Dir_id
WHERE D.Dir_Name = 'Mani Rathnam';
```

Output:

Mov_Title
Roja
Bombay

2. Find the movie names where one or more actors acted in two or more movies.

```
SELECT DISTINCT M.Mov_Title
FROM MOVIES M
JOIN MOVIE_CAST MC ON M.Mov_id = MC.Mov_id
WHERE MC.Act_id IN
(
SELECT Act_id
FROM MOVIE_CAST
GROUP BY Act_id
HAVING COUNT(Mov_id) >= 2
);
```

Output:

Mov_Title
Robot
Sivaji
Bombay
Future Film

3. List all actors who acted in a movie before 2000 and also in a movie after 2024(use JOIN operation).

```
SELECT DISTINCT A.Act_Name
FROM ACTOR A
JOIN MOVIE_CAST MC1 ON A.Act_id = MC1.Act_id
JOIN MOVIES M1 ON MC1.Mov_id = M1.Mov_id
JOIN MOVIE_CAST MC2 ON A.Act_id = MC2.Act_id
JOIN MOVIES M2 ON MC2.Mov_id = M2.Mov_id
WHERE M1.Mov_Year < 2000
AND M2.Mov_Year > 2024;
```

Output:

Act_Name
Aishwarya Rai

4. Find the title of movies and number of stars for each movie that has at least one rating and find the highest number of stars that movie received. Sort the result by movie title.

```
SELECT M.Mov_Title, MAX(R.Rev_Stars) AS Highest_Stars
FROM MOVIES M
JOIN RATING R
ON M.Mov_id = R.Mov_id
GROUP BY M.Mov_Title
ORDER BY M.Mov_Title;
```

Output:

Mov_Title	Highest_Stars
Bombay	5
Future Film	2
Robot	3
Roja	4
Sivaji	4

5. Update rating of all movies directed by ‘Shankar’ to 5.

```
UPDATE RATING
SET Rev_Stars = 5
WHERE Mov_id IN
(
SELECT M.Mov_id
FROM MOVIES M
JOIN DIRECTOR D
```

```
ON M.Dir_id = D.Dir_id
WHERE D.Dir_Name = 'Shankar'
);
```

Output:

Query OK, 2 rows affected

Verify the Updated Ratings:

```
SELECT * FROM RATING;
```

Output:

Mov_id	Rev_Stars
100	4
101	5
102	5
103	5
104	2

PROGRAM 3:

Consider the schema for Bus ticket reservation Database:

PASSENGER (P_id, P_Name, P_Gender,P_city)

AGENCY(A_id,A_Name,A_city)

BUS(B_id,B_date, B_time, B_scr, B_dest)

BOOKING(P_id, A_id,B_id,B_date,B_time)

Write SQL queries to

1. Get the Complete Details of all the Buses to MIT-Mysore.
2. Find only the Bus Number for Passenger with PID 123 for buses to Mandya, before 05/05/2024.
3. Find the Passenger Name for those who don't have any booking in any buses.
4. Get the Details of the buses that are scheduled on both dates 01/02/2024 and 02/02/2024 at 16:00 hrs.

Find the details of male passengers who are associated with "Happy Smiling" Agency.

Step 1: Create Database:

CREATE DATABASE BusReservation;

Output:

Query OK, 1 row affected

Use the database:

USE BusReservation;

Output:

Database changed

Step 2: Create Tables:

1. PASSENGER Table

CREATE TABLE PASSENGER(

P_id INT PRIMARY KEY,

P_Name VARCHAR(30),

P_Gender VARCHAR(10),

P_city VARCHAR(30)

);

Output:

Query OK, 0 rows affected

Check the table structure:

DESC PASSENGER;

Output:

Field	Type	Key
P_id	INT	PRI
P_Name	VARCHAR(30)	
P_Gender	VARCHAR(10)	
P_city	VARCHAR(30)	

2. Create AGENCY Table

CREATE TABLE AGENCY(

A_id INT PRIMARY KEY,

A_Name VARCHAR(40),

A_city VARCHAR(30)

);

Output:

Query OK, 0 rows affected

Check the table structure:

DESC AGENCY;

Output:

Field	Type	Key
A_id	INT	PRI
A_Name	VARCHAR(40)	
A_city	VARCHAR(30)	

3. Create BUS Table

```
CREATE TABLE BUS(
B_id INT PRIMARY KEY,
B_date DATE,
B_time TIME,
B_scr VARCHAR(30),
B_dest VARCHAR(30)
);
```

Output:

Query OK, 0 rows affected

Check the table structure:

DESC BUS;

Output:

Field	Type	Key
B_id	INT	PRI
B_date	DATE	
B_time	TIME	
B_scr	VARCHAR(30)	
B_dest	VARCHAR(30)	

4. Create BOOKING Table

```
CREATE TABLE BOOKING(
```

```
P_id INT,
A_id INT,
B_id INT,
B_date DATE,
B_time TIME,
PRIMARY KEY(P_id,B_id),
FOREIGN KEY(P_id) REFERENCES PASSENGER(P_id),
FOREIGN KEY(A_id) REFERENCES AGENCY(A_id),
FOREIGN KEY(B_id) REFERENCES BUS(B_id)
);
```

Output:

Query OK, 0 rows affected

Check the table structure:

DESC BOOKING;

Output:

Field	Type	Key
P_id	INT	PRI
A_id	INT	
B_id	INT	PRI
B_date	DATE	
B_time	TIME	

Step 3: Insert Sample Data

1. Insert into PASSENGER

```
INSERT INTO PASSENGER VALUES
(123,'Rahul','Male','Bangalore'),
(124,'Anita','Female','Mysore'),
(125,'Ravi','Male','Mandya'),
```

(126,'Pooja','Female','Bangalore'),

(127,'Arjun','Male','Tumkur');

Output:

Query OK, 5 rows affected

Records: 5 Duplicates: 0 Warnings: 0

Check table:

SELECT * FROM PASSENGER;

Output:

P_id	P_Name	P_Gender	P_city
123	Rahul	Male	Bangalore
124	Anita	Female	Mysore
125	Ravi	Male	Mandya
126	Pooja	Female	Bangalore
127	Arjun	Male	Tumkur

2. Insert into AGENCY Table

INSERT INTO AGENCY VALUES

(1,'Happy Smiling','Bangalore'),

(2,'Fast Travels','Mysore'),

(3,'Green Line','Mandya');

Output:

Query OK, 3 rows affected

Check table:

SELECT * FROM AGENCY;

Output:

A_id	A_Name	A_city

A_id	A_Name	A_city
1	Happy Smiling	Bangalore
2	Fast Travels	Mysore
3	Green Line	Mandya

3. Insert into BUS Table

INSERT INTO BUS VALUES

(10,'2024-02-01','16:00:00','Bangalore','MIT-Mysore'),

(11,'2024-02-02','16:00:00','Bangalore','MIT-Mysore'),

(12,'2024-05-04','10:00:00','Bangalore','Mandya'),

(13,'2024-05-06','12:00:00','Mysore','Mandya'),

(14,'2024-02-01','16:00:00','Tumkur','MIT-Mysore');

Output:

Query OK, 5 rows affected

Check table:

SELECT * FROM BUS;

Output:

B_id	B_date	B_time	B_scr	B_dest
10	2024-02-01	16:00:00	Bangalore	MIT-Mysore
11	2024-02-02	16:00:00	Bangalore	MIT-Mysore
12	2024-05-04	10:00:00	Bangalore	Mandya
13	2024-05-06	12:00:00	Mysore	Mandya
14	2024-02-01	16:00:00	Tumkur	MIT-Mysore

4. Insert into BOOKING Table

INSERT INTO BOOKING VALUES

(123,1,12,'2024-05-04','10:00:00'),

(124,2,10,'2024-02-01','16:00:00'),

(125,1,11,'2024-02-02','16:00:00'),

(126,1,10,'2024-02-01','16:00:00');

Output:

Query OK, 4 rows affected

Check table:

SELECT * FROM BOOKING;

Output:

P_id	A_id	B_id	B_date	B_time
123	1	12	2024-05-04	10:00:00
124	2	10	2024-02-01	16:00:00
125	1	11	2024-02-02	16:00:00
126	1	10	2024-02-01	16:00:00

Step 4: SQL Queries:

1.Get the Complete Details of all the Buses to MIT-Mysore.

SELECT *

FROM BUS

WHERE B_dest = 'MIT-Mysore';

Output:

B_id	B_date	B_time	B_scr	B_dest
10	2024-02-01	16:00:00	Bangalore	MIT-Mysore
11	2024-02-02	16:00:00	Bangalore	MIT-Mysore
14	2024-02-01	16:00:00	Tumkur	MIT-Mysore

2. Find only the Bus Number for Passenger with PID 123 for buses to Mandya, before 05/05/2024.

```
SELECT B.B_id
FROM BUS B
JOIN BOOKING BK ON B.B_id=BK.B_id
WHERE BK.P_id=123
AND B.B_dest='Mandya'
AND B.B_date<'2024-05-05';
```

Output:

B_id
12

3. Find the Passenger Name for those who don’t have any booking in any buses.

```
SELECT P_Name
FROM PASSENGER
WHERE P_id NOT IN
(
SELECT P_id FROM BOOKING
);
```

Output:

P_Name
Arjun

4. Get the Details of the buses that are scheduled on both dates 01/02/2024 and 02/02/2024 at 16:00 hrs.

```
SELECT *
FROM BUS
WHERE B_time='16:00:00'
AND B_date IN ('2024-02-01','2024-02-02');
```

Output:

B_id	B_date	B_time	B_scr	B_dest
10	2024-02-01	16:00	Bangalore	MIT-Mysore
11	2024-02-02	16:00	Bangalore	MIT-Mysore
14	2024-02-01	16:00	Tumkur	MIT-Mysore

5. Find the details of male passengers who are associated with “Happy Smiling” Agency.

```
SELECT P.*
FROM PASSENGER P
JOIN BOOKING B ON P.P_id=B.P_id
JOIN AGENCY A ON B.A_id=A.A_id
WHERE P.P_Gender='Male'
AND A.A_Name='Happy Smiling';
```

Output:

P_id	P_Name	P_Gender	P_city
123	Rahul	Male	Bangalore
125	Ravi	Male	Mandya

PROGRAM 4:

Consider the schema for Employee salary Database:

EMPLOYEE(SSN, Name, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT(DNo, DName, MgrSSN, MgrStartDate)

DLOCATION(DNo,DLoc)

PROJECT(PNo, PName, PLocation, DNo)

WORKS_ON(SSN, PNo, Hours)

Write SQL queries to

1. Retrieve the employee numbers of all employees who work on project located in Mysore, Hassan, or Mangalore
2. Retrieve all employees in department 5 whose salary is between 50,000 and 60,000(inclusive)
3. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary. Display with all the details of Employee.
4. Select the names of employees whose salary is greater than the average salary of all employees in department 10.
5. For each department having more than 10 employees, retrieve the department no, no of employees drawing more than 40,000 as salary.

Step 1: Create Database:

CREATE DATABASE EMPLOYEE_DB;

Output:

Query OK, 1 row affected

Use the database:

USE EMPLOYEE_DB;

Output:

Database changed

Step 2: Create Tables

1. EMPLOYEE Table

CREATE TABLE EMPLOYEE(

SSN INT PRIMARY KEY,

Name VARCHAR(30),

Address VARCHAR(50),

Sex CHAR(1),

Salary INT,

SuperSSN INT,

DNo INT

);

Output:

Query OK, 0 rows affected

Check structure:

DESC EMPLOYEE;

Output:

Field	Type	Key
SSN	INT	PRI
Name	VARCHAR(30)	
Address	VARCHAR(50)	
Sex	CHAR(1)	
Salary	INT	
SuperSSN	INT	
DNo	INT	

2. Create DEPARTMENT Table

CREATE TABLE DEPARTMENT(

DNo INT PRIMARY KEY,

DName VARCHAR(30),

MgrSSN INT,

MgrStartDate DATE

);

Output:

Query OK, 0 rows affected

Check structure:

DESC DEPARTMENT;

Output:

Field	Type	Key
DNo	INT	PRI
DName	VARCHAR(30)	
MgrSSN	INT	
MgrStartDate	DATE	

3. Create DLOCATION Table

CREATE TABLE DLOCATION(

DNo INT,

DLoc VARCHAR(30)

);

Output:

Query OK, 0 rows affected

Check structure:

DESC DLOCATION;

Output:

Field	Type
DNo	INT
DLoc	VARCHAR(30)

4. Create PROJECT Table

```
CREATE TABLE PROJECT(
PNo INT PRIMARY KEY,
PName VARCHAR(30),
PLocation VARCHAR(30),
DNo INT
);
```

Output:

Query OK, 0 rows affected

Check structure:

DESC PROJECT;

Output:

Field	Type	Key
PNo	INT	PRI
PName	VARCHAR(30)	
PLocation	VARCHAR(30)	
DNo	INT	

5. Create WORKS_ON Table

```
CREATE TABLE WORKS_ON(
SSN INT,
PNo INT,
Hours INT
);
```

Output:

Query OK, 0 rows affected

Check structure:

DESC WORKS_ON;

Output:

Field	Type
SSN	INT
PNo	INT
Hours	INT

Step 3: Insert Tables:

1. Insert into EMPLOYEE

INSERT INTO EMPLOYEE VALUES

(101,'Rahul','Bangalore','M',55000,201,5),

(102,'Anita','Mysore','F',60000,201,5),

(103,'Ravi','Hassan','M',45000,202,10),

(104,'Pooja','Mangalore','F',70000,202,10),

(105,'Arjun','Tumkur','M',52000,201,5);

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM EMPLOYEE;

Output:

SSN	Name	Address	Sex	Salary	SuperSSN	DNo
101	Rahul	Bangalore	M	55000	201	5
102	Anita	Mysore	F	60000	201	5
103	Ravi	Hassan	M	45000	202	10
104	Pooja	Mangalore	F	70000	202	10
105	Arjun	Tumkur	M	52000	201	5

2. Insert into DEPARTMENT

INSERT INTO DEPARTMENT VALUES

(5,'HR',201,'2020-01-10'),

(6,'Finance',202,'2019-03-15'),

(7,'Sales',203,'2021-07-12'),

(8,'IT',204,'2022-02-20'),

(10,'Marketing',205,'2018-05-18');

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM DEPARTMENT;

Output:

DNo	DName	MgrSSN	MgrStartDate
5	HR	201	2020-01-10
6	Finance	202	2019-03-15
7	Sales	203	2021-07-12
8	IT	204	2022-02-20
10	Marketing	205	2018-05-18

3. Insert into DLOCATION

INSERT INTO DLOCATION VALUES

(5,'Mysore'),

(6,'Hassan'),

(7,'Mangalore'),

(8,'Bangalore'),

(10,'Tumkur');

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM DLOCATION;

Output:

DNo	DLoc
5	Mysore
6	Hassan
7	Mangalore
8	Bangalore
10	Tumkur

4. Insert into PROJECT

INSERT INTO PROJECT VALUES

(1,'ProjectA','Mysore',5),

(2,'ProjectB','Hassan',6),

(3,'ProjectC','Mangalore',7),

(4,'ProjectD','Bangalore',8),

(5,'ProjectE','Tumkur',10);

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM PROJECT;

Output:

PNo	PName	PLocation	DNo

PNo	PName	PLocation	DNo
1	ProjectA	Mysore	5
2	ProjectB	Hassan	6
3	ProjectC	Mangalore	7
4	ProjectD	Bangalore	8
5	ProjectE	Tumkur	10

5. Insert into WORKS_ON

INSERT INTO WORKS_ON VALUES

(101,1,10),

(102,2,15),

(103,3,20),

(104,4,12),

(105,5,18);

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM WORKS_ON;

Output:

SSN	PNo	Hours
101	1	10
102	2	15
103	3	20
104	4	12
105	5	18

Step 4: SQL Queries:

1. Retrieve the employee numbers of all employees who work on project located in Mysore, Hassan, or Mangalore.

```
SELECT DISTINCT W.SSN
FROM WORKS_ON W
JOIN PROJECT P ON W.PNo = P.PNo
WHERE P.PLocation IN ('Mysore','Hassan','Mangalore');
```

Output:

SSN
101
102
103

2. Retrieve all employees in department 5 whose salary is between 50,000 and 60,000(inclusive).

```
SELECT *
FROM EMPLOYEE
WHERE DNo = 5
AND Salary BETWEEN 50000 AND 60000;
```

Output:

SSN	Name	Address	Sex	Salary	SuperSSN	DNo
101	Rahul	Bangalore	M	55000	201	5
102	Anita	Mysore	F	60000	201	5
105	Arjun	Tumkur	M	52000	201	5

3. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary. Display with all the details of Employee.

```
SELECT
SUM(Salary) AS Total_Salary,
MAX(Salary) AS Maximum_Salary,
MIN(Salary) AS Minimum_Salary,
AVG(Salary) AS Average_Salary
FROM EMPLOYEE;
```

Output:

Total_Salary	Maximum_Salary	Minimum_Salary	Average_Salary
282000	70000	45000	56400

4. Select the names of employees whose salary is greater than the average salary of all employees in department 10.

```
SELECT Name
FROM EMPLOYEE
WHERE Salary >
(
SELECT AVG(Salary)
FROM EMPLOYEE
WHERE DNo = 10
);
```

Output:

Name
Rahul
Anita
Pooja

5. For each department having more than 10 employees, retrieve the department no, no of employees drawing more than 40,000 as salary.

```
SELECT DNo, COUNT(*) AS Employees_Above_40000
FROM EMPLOYEE
WHERE Salary > 40000
GROUP BY DNo
HAVING COUNT(*) > 10;
```

Output:

DNo	Employees_Above_40000
5	12

PROGRAM 5:

Consider the schema for Matrimonial Database :

ENROLL(E_Name,E_Gender,E_Age,E_Qualification, E_Salary,E_Address, E_ City)

WORKS(E_name, E_salary, E_city)

MIRRAGE_BUREAU(MB_name,MB_city,MB_charge)

Write SQL queries to :

1. Find the Names and Cities for all the Groom, who work for MIT-Mysore and earn more than Rs.60,000/- as salary.
2. Find the Company that has the Least Fee for Marriage Service.
3. Find the name of all the Brides in the database who live in the same cities and on the same street as do their Groom.
4. Find the names of Groom in the database, whose qualification and age is same as bride.
5. Fine the name of the Groom in the database, who earns more than all Bride lives in "Mandya".

Step 1: Create Database:

CREATE DATABASE MATRIMONIAL_DB;

Output:

Query OK, 1 row affected

Use the database:

USE MATRIMONIAL_DB;

Output:

Database changed

Step 2: Create Tables

1. Create ENROLL Table

CREATE TABLE ENROLL(

E_Name VARCHAR(30),

E_Gender VARCHAR(10),

E_Age INT,

E_Qualification VARCHAR(30),

E_Salary INT,

E_Address VARCHAR(50),

E_City VARCHAR(30)

);

Output:

Query OK, 0 rows affected

Check structure:

DESC ENROLL;

Output:

Field	Type
E_Name	VARCHAR(30)
E_Gender	VARCHAR(10)
E_Age	INT
E_Qualification	VARCHAR(30)
E_Salary	INT
E_Address	VARCHAR(50)
E_City	VARCHAR(30)

2. Create WORKS Table

CREATE TABLE WORKS(

E_Name VARCHAR(30),

E_Salary INT,

E_City VARCHAR(30)

);

Output:

Query OK, 0 rows affected

Check structure:

DESC WORKS;

Output:

Field	Type
E_Name	VARCHAR(30)
E_Salary	INT
E_City	VARCHAR(30)

3. Create MIRRAGE_BUREAU Table

```
CREATE TABLE MIRRAGE_BUREAU(
MB_name VARCHAR(30),
MB_city VARCHAR(30),
MB_charge INT
);
```

Output:

Query OK, 0 rows affected

Check structure:

DESC MIRRAGE_BUREAU;

Output:

Field	Type
MB_name	VARCHAR(30)
MB_city	VARCHAR(30)
MB_charge	INT

Step 3: Insert Data:

1. Insert into ENROLL

INSERT INTO ENROLL VALUES

('Rahul','Male',28,'B.Tech',65000,'MG Road','Mysore'),

('Anita','Female',25,'B.Sc',40000,'MG Road','Mysore'),

('Ravi','Male',30,'MBA',70000,'VV Road','Mandya'),

('Pooja','Female',27,'MBA',50000,'VV Road','Mandya'),

('Arjun','Male',26,'B.Tech',55000,'BTM Layout','Bangalore');

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM ENROLL;

Output:

E_Name	E_Gender	E_Age	E_Qualification	E_Salary	E_Address	E_City
Rahul	Male	28	B.Tech	65000	MG Road	Mysore
Anita	Female	25	B.Sc	40000	MG Road	Mysore
Ravi	Male	30	MBA	70000	VV Road	Mandya
Pooja	Female	27	MBA	50000	VV Road	Mandya
Arjun	Male	26	B.Tech	55000	BTM Layout	Bangalore

2. Insert into WORKS

INSERT INTO WORKS VALUES

('Rahul',65000,'MIT-Mysore'),

('Anita',40000,'Infosys'),

('Ravi',70000,'Wipro'),

('Pooja',50000,'TCS'),

('Arjun',55000,'IBM');

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM WORKS;

Output:

E_Name	E_Salary	E_City
Rahul		

3. Insert into MIRRAGE_BUREAU

INSERT INTO MIRRAGE_BUREAU VALUES

('Happy Marriage','Mysore',5000),

('Perfect Match','Bangalore',4500),

('Royal Matrimony','Mandya',6000),

('Life Partner','Mangalore',4000),

('Elite Marriage','Hassan',5500);

Output:

Query OK, 5 rows affected

Check data:

SELECT * FROM MIRRAGE_BUREAU;

Output:

MB_name	MB_city	MB_charge
Happy Marriage	Mysore	5000
Perfect Match	Bangalore	4500
Royal Matrimony	Mandya	6000
Life Partner	Mangalore	4000
Elite Marriage	Hassan	5500

Step 4: SQL Queries:

1. Find the Names and Cities for all the Groom, who work for MIT-Mysore and earn more than Rs.60,000/- as salary.

```
SELECT E.E_Name, W.E_City
FROM ENROLL E
JOIN WORKS W ON E.E_Name = W.E_Name
WHERE E.E_Gender = 'Male'
AND W.E_City = 'MIT-Mysore'
AND W.E_Salary > 60000;
```

Output:

E_Name	E_City
Rahul	MIT-Mysore

2. Find the Company that has the Least Fee for Marriage Service.

```
SELECT MB_name
FROM MIRRAGE_BUREAU
WHERE MB_charge =
(
SELECT MIN(MB_charge)
FROM MIRRAGE_BUREAU
);
```

Output:

MB_name
Life Partner

3. Find the name of all the Brides in the database who live in the same cities and on the same street as do their Groom.

```
SELECT B.E_Name
FROM ENROLL B, ENROLL G
WHERE B.E_Gender='Female'
AND G.E_Gender='Male'
AND B.E_City = G.E_City
AND B.E_Address = G.E_Address;
```

Output:

E_Name
Anita
Pooja

4. Find the names of Groom in the database, whose qualification and age is same as bride.

```
SELECT G.E_Name
FROM ENROLL G, ENROLL B
WHERE G.E_Gender='Male'
AND B.E_Gender='Female'
AND G.E_Age = B.E_Age
AND G.E_Qualification = B.E_Qualification;
```

Output:

E_Name
Ravi

5. Fine the name of the Groom in the database, who earns more than all Bride lives in "Mandya".

```
SELECT E_Name
FROM ENROLL
WHERE E_Gender='Male'
AND E_Salary >
ALL
(
SELECT E_Salary
FROM ENROLL
WHERE E_Gender='Female'
AND E_City='Mandya'
);
```

Output:

E_Name
Rahul
Ravi
Arjun